РОЗРОБКА ЗАСТОСУНКІВ ЗІ ШТУЧНИМ ІНТЕЛЕКТОМ

Практична робота №2. Локальний inference спеціалізованої моделі

Мета: Запустити локальний inference готової спеціалізованої моделі (object detection) у веб-застосунку: отримати структурований результат із confidence, відфільтрувати за порогом, виміряти час виконання, винести inference в окремий модуль і зібрати перші приклади для оцінювання якості. Мова — Python (Ultralytics YOLO, FastAPI).

Зміст

1. Теоретичні відомості
2. Завдання до виконання
3. Критерії приймання
4. Контрольні запитання
5. Рекомендована література

1. Теоретичні відомості

У ПР1 «двигуном» був зовнішній REST API. Тут AI виконується **локально** — готовою моделлю object detection на вашій машині, без звернення до зовнішнього сервісу. Ви завантажите зображення й отримаєте знайдені об'єкти, а заразом зберете перші приклади для оцінювання якості, які знадобляться далі.

Опора, щоб виконувати роботу свідомо.

Inference проти навчання. Ви не навчаєте модель, а лише запускаєте готову (pretrained): подаєте вхід і отримуєте передбачення — це і є inference. Навчання вже виконане, ваги моделі фіксовані.

Локально проти хмарного API. Inference можна виконувати на своїй машині або звертатися до хмарного сервісу. Локально — дані не покидають машину, немає плати за виклик і мережевої затримки, працює офлайн, але потрібні власні обчислювальні ресурси й модель має вміститися в них. Це компроміс, який ви оцінюєте під задачу (пригадайте огляд з ЛК1).

Що повертає детектор. Object detection для кожного знайденого об'єкта дає три речі: **клас** (що це), **рамку** (де це — координати прямокутника) і **впевненість** (confidence — наскільки модель певна, від 0 до 1).

Поріг упевненості. Не всі передбачення однаково надійні. Відсікаючи ті, що нижче порога confidence, ви керуєте компромісом: високий поріг — менше хибних спрацювань, але більше пропусків; низький — навпаки. «Правильного» порога немає, він залежить від задачі.

Вартість завантаження моделі. Завантаження ваг у пам'ять — дорога операція. Тому модель ініціалізують **один раз** і перевикористовують для всіх запитів, а не завантажують щоразу.

Оцінювання якості. Щоб судити, чи добре працює модель, самих передбачень мало — потрібні приклади з відомою правильною відповіддю, з якими можна порівняти. Типові помилки детектора: пропуск об'єкта, хибне спрацювання, неправильний клас, дубль.

2. Завдання до виконання

Зробіть веб-застосунок, який приймає зображення й показує знайдені на ньому об'єкти: для кожного — клас, рамку (координати) і впевненість (confidence), а також їхню кількість і час виконання. Модель — **готова pretrained-модель**, inference — локальний. Бекенд — на FastAPI; роботу з моделлю винесіть в окремий модуль.

Спосіб реалізації — на ваш розсуд (самостійно або скеровуючи AI-асистента), але ви маєте **розуміти й уміти пояснити** кожне рішення.

Матеріали. В архіві до цієї роботи, папка `pr2`, вас чекає підготовлене середовище: `requirements.txt` із залежностями, скрипт `check_env.py`, каркас застосунку в папці `app` і папка `eval` для набору, на якому оцінюватимете якість.

Каркас дає структуру проєкту й веб-рівень, який уже вміє приймати завантажений файл. Реалізації в ньому немає: `load_model` і `detect` — заготовки, обробки збоїв немає, і саме це ви й пишете.

`check_env.py` варто запустити **заздалегідь, а не на занятті**: крім пакетів він завантажує ваги моделі з мережі, і цей разовий крок триває кілька десятків секунд.

Де вести роботу. Розпакуйте архів у корінь власного репозиторію курсу — поруч із папкою `pr1` від попередньої роботи. Свій код пишіть у папці роботи й комітьте частинами.

Орієнтири.

- Готова модель детекції: **Ultralytics YOLO** (напр. вага `yolov8n` — легка, працює на CPU; завантажується автоматично при першому використанні). Результат моделі для кожного об'єкта містить клас, координати рамки й confidence.
- Доречні бібліотеки: `ultralytics` (модель), `FastAPI` + `uvicorn` (веб-рівень), засоби приймання завантаженого файлу.

Продумайте самостійно.

- Яким зробити поріг упевненості й чи давати користувачеві його змінювати?
- Де в застосунку модель має завантажуватися, щоб це сталося один раз, а не на кожен запит?
- Де межа між кодом inference (робота з моделлю) і рештою застосунку? Що веб-рівень має знати про модель?
- Як виміряти час inference і що на нього впливає?
- Як повестися, коли надійшов некоректний файл (не зображення, порожній)?

Оцінювання якості (перші приклади). Заготовку для цього — папку `eval` з описом і шаблоном `expected.json` — знайдете в архіві. Підготуйте 5–7 зображень із відомим вмістом і запишіть, які об'єкти на них очікуєте. Прогнавши детектор, порівняйте результат з очікуваним і зафіксуйте типи помилок: пропуски, хибні спрацювання, неправильний клас, дублі. Поспостерігайте, як поріг упевненості змінює картину. Цей набір і спостереження — заготовка для систематичного оцінювання в наступних роботах.

3. Критерії приймання

- Використано **готову pretrained-модель**; inference виконується **локально**, без зовнішнього API.

- Результат **структурований**: клас, координати рамки й confidence для кожного об'єкта.
- Реалізовано **фільтрацію за порогом** упевненості.
- **Час inference** вимірюється й показується.
- Робота з моделлю винесена в **окремий модуль**; модель завантажується **один раз**, а не на кожен запит.
- Веб-ендпоінт приймає зображення й повертає результат як **JSON** або **відповідний HTTP-статус** за помилки; сторінка показує знайдені об'єкти.
- Сформовано **перші приклади для оцінювання** (≥5) і коротко проаналізовано помилки моделі.
- Ви можете **пояснити** кожне ухвалене рішення.

4. Контрольні запитання

1. Чим локальний inference відрізняється від виклику хмарного AI API — переваги й компроміси?
2. Що таке confidence і як поріг упевненості впливає на пропуски проти хибних спрацювань?
3. Чому модель варто завантажувати один раз?
4. Навіщо застосунку структурований результат детекції (а не «сира» відповідь моделі)?
5. Навіщо вимірювати час inference і від чого він залежить?
6. Які типи помилок робить детектор і як їх виявити?
7. Навіщо вже зараз збирати приклади для оцінювання якості?

5. Рекомендована література

1. Ultralytics YOLO Documentation. URL: https://docs.ultralytics.com/ (дата звернення: 01.09.2026).
2. FastAPI Documentation. URL: https://fastapi.tiangolo.com/ (дата звернення: 01.09.2026).
3. Lin T.-Y., Maire M., Belongie S. et al. Microsoft COCO: Common Objects in Context. arXiv:1405.0312. URL: https://arxiv.org/abs/1405.0312 (дата звернення: 01.09.2026).